



UNIVERSIDADE DO MINHO

LICENCIATURA EM ENGENHARIA INFORMÁTICA

Sistemas Operativos

Grupo 36

Roberto Capela (A100655)

José Lima (A106888)

Rodrigo Silva (A108961)

17 de maio de 2025

Conteúdo

1	Introdução	3
2	Arquitetura do Sistema	3
3	Funcionalidades Implementadas	3
4	Avaliação Experimental	4
5	Dificuldades Sentidas	5
6	Conclusão	6

1 Introdução

Este relatório descreve o desenvolvimento de um serviço de indexação e pesquisa de documentos de texto, realizado no âmbito da unidade curricular de Sistemas Operativos. O objetivo do projeto é permitir que utilizadores indexem, consultem, removam e pesquisem documentos armazenados localmente, através de uma arquitetura cliente-servidor implementada em C, comunicando via *pipes* nomeados. O relatório detalha a arquitetura do sistema, as funcionalidades implementadas, as principais dificuldades sentidas e uma avaliação experimental focada na pesquisa concorrente.

2 Arquitetura do Sistema

O sistema é constituído por dois programas principais: **dclient** (cliente) e **dserver** (servidor).

- **Cliente (dclient):** Interface de linha de comandos usada pelo utilizador para submeter operações ao servidor (indexação, consulta, remoção, pesquisa), recebendo e apresentando os resultados.

Este recebe o *input* do utilizador, verifica-o e cria o pedido de 512 *bytes* a enviar ao servidor. O pedido segue sempre a mesma estrutura: 3 *bytes* para o PID do cliente para onde o servidor irá encaminhar a resposta, 1 *byte* que identifica o tipo de pedido seguido de um *byte* nulo, que separa este *header* dos restantes parâmetros do pedido. Estes por último são adicionados como um *array* de *strings*.

De seguida, o pedido é enviado para o FIFO do servidor e é criado um FIFO para a resposta, baseado no PID do cliente, onde o mesmo aguarda o *reply* do servidor.

- **Servidor (dserver):** Responsável por gerir a meta-informação dos documentos (identificador, título, autores, ano, caminho), processar pedidos dos clientes, executar pesquisas e garantir a persistência dos dados.

O servidor inicia as suas estruturas como o FIFO onde recebe os pedidos e verifica se existe dados para carregar de um ficheiro persistente guardado em disco e fica à espera de pedidos dos clientes. No final, caso tenha sido encerrado com a chegada de um pedido do tipo **-f**, ele retorna os dados em memória novamente para o ficheiro em disco e termina.

3 Funcionalidades Implementadas

- **Indexação de documentos:** O utilizador pode indexar (-a) um documento fornecendo título, autores, ano e caminho relativo para o documento. O servidor atribui e retorna um identificador único.
- **Consulta e remoção:** Permite consultar (-c) ou remover (-d) a meta-informação de um documento pelo seu identificador.
- **Pesquisa no conteúdo:**
 - Número de linhas que contêm uma palavra-chave (-l).
 - Listagem de documentos que contêm uma palavra-chave (-s).
 - Pesquisa concorrente: possibilidade de definir o número máximo de processos para a pesquisa (-s *keyword nr_processes*).
- **Persistência:** O comando -f permite encerrar o servidor de forma controlada, onde a meta-informação dos documentos é guardada em ficheiro persistente, sendo recarregada no arranque do servidor.

4 Avaliação Experimental

Foram realizados testes para avaliar o ganho de desempenho na pesquisa de documentos (-s) com diferentes números de processos concorrentes.

Metodologia

- **Ambiente de testes:** Os testes foram realizados numa máquina com as seguintes características:
 - Processador: 12th Gen Intel® Core™ i5-12400F @ 2.50 GHz
 - Memória RAM: 16 GB
 - Sistema operativo: Windows 11 com WSL
- **Conjunto de documentos:** Foram utilizados conjuntos com 100, 500, 1000 e 1500 documentos de texto, com tamanhos e conteúdos variados. Os testes apresentados referem-se ao conjunto com 1000 documentos, dado que os restantes apresentaram padrões semelhantes.
- **Procedimento:** Mediu-se o tempo de execução da pesquisa para diferentes números de processos (1, 2, 4, 8, ...), mantendo a palavra-chave e constantes.

Resultados

Nº Processos	Tempo (ms)	Speedup	Eficiência (%)
1	2060.79	1.0	100
2	953.39	2.16	108
4	516.77	3.98	99.5
8	480.52	4.28	53.5
16	424.73	4.85	30.3125
32	363.11	5.67	17.71875
64	370.38	5.56	8.6875
128	392.74	5.24	4.09375

Tabela 4.1: Resultados da pesquisa concorrente para a palavra [praia] em 1000 documentos

Speedup: $S_p = \frac{T_{sequencial}}{T_{paralelo}}$

Eficiência: $E_p = \frac{S_p}{p} \times 100$

Análise

A análise dos resultados obtidos na pesquisa concorrente (-s "palavra" N) revela um comportamento típico de escalonamento até um ponto ótimo, seguido de degradação de desempenho. Inicialmente, à medida que o número de processos usados aumenta (de 1 até 32), o tempo de execução da pesquisa diminui de forma significativa, devido à distribuição eficiente do trabalho pelos processos filhos e à boa gestão dos recursos pelo sistema operativo. No entanto, a partir de 64 processos, verifica-se um aumento no tempo de execução, sinalizando o ponto de saturação da paralelização. Esse aumento ocorre porque o custo de criar, gerir e agendar um número elevado de processos começa a superar os ganhos obtidos com a divisão do trabalho, ou seja, o sistema passa a gastar mais tempo a coordenar os processos do que a executar efetivamente a pesquisa. Este comportamento evidencia que existe um ponto ótimo de paralelismo (neste caso entre 16 e 32 processos), após o qual o desempenho degrada devido ao overhead de concorrência. Assim, mais processos não implicam necessariamente em maior eficiência, sendo essencial calibrar este parâmetro consoante a carga e os recursos disponíveis.

5 Dificuldades Sentidas

A principal dificuldade sentida foi a implementação da pesquisa concorrente (-s com múltiplos processos). Foi necessário garantir a correta criação e agregação dos resultados parciais sem recorrer a ferramentas externas de paralelização. A gestão da comunicação entre processos e a recolha dos resultados para o cliente revelou-se desafiante, especialmente para garantir que todos os processos terminam corretamente. Assim como a falha da implementação de funcionalidade *caching* que limita a meta-informação guardada em memória.

6 Conclusão

O trabalho permitiu consolidar conhecimentos em programação de sistemas, comunicação entre processos e gestão eficiente de recursos. A implementação da pesquisa concorrente demonstrou ganhos de desempenho claros, embora limitada pela sobrecarga de criação de processos e pelas características do hardware. Fica como proposta futura a implementação de *caching* em memória, conforme previsto no enunciado, para otimizar e controlar a utilização de memória.